

LEBANESE
INTERNATIONAL
UNIVERSITY



Ma7ali - Mobile Application

Senior Project

By

Naji Al-Jrwan

Submitted to the School of Arts & Science of the
Lebanese International University **Campus**, Lebanon

In part of fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE IN
COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

Supervised by: Mr. Fouad Najm

Spring 2024-2025

Dedication:

I wholeheartedly dedicate this project to my esteemed professor, **Mr. Fouad Najm**, whose unwavering support, insightful guidance, and generous encouragement have been instrumental from the very first day to the successful completion of this work. His dedication to excellence, modern perspective, and constant motivation played a pivotal role in shaping this project into a practical and efficient solution. I remain deeply grateful for his mentorship and invaluable contributions at every stage of this journey.

Acknowledgment:

First and foremost, I would like to express my sincere gratitude to **Mr. Fouad Najm**, whose continuous support, expert advice, and constructive feedback have been crucial to the successful realization of this project. His commitment to academic excellence and innovation greatly inspired and guided my work.

I also extend my appreciation to the faculty and staff of the **Lebanese International University (LIU)** for providing a supportive learning environment and access to the necessary resources throughout the development of this project.

Lastly, I am thankful to my family and friends for their constant encouragement, patience, and belief in me, especially during challenging moments. Their moral support has been a source of strength and motivation.

Abstract:

As a developer, my role goes beyond building websites or applications—it's about solving **real-world problems** and making everyday **tasks** easier through technology.

The idea for **Ma7ali** application came from something simple: every time I visited small local shops or mini markets in villages like Ghazzah, Alkhyara and many shops across all of Lebanon, I noticed how owners manually recorded sales, customer debts, and inventory on paper.

That's when I thought—why not create an app that simplifies all of this? An app that not only helps shop owners track sales, inventory, and customer debts, but also supports them in growing their business with modern, efficient, and easy-to-use tools.

Throughout this report, I will explain every feature and tool in this mobile application in details to highlight how does it solve the problem mentioned above and how does it make it easier for shop owners to manage their store in terms of customers, products and sales and how does it help them grow their business through some modern analytics of the sales and inventory of their store.

Table of Content:

Chapter 1: Project Theory & Foundation

1.1 - Introduction:	8
1.2 - Objective:	8
1.3 - scope:	8
1.4 - Technology constraints:	9
Softwares:	9
Languages:	9
Frontend Technologies:	9
Backend / Cloud Services (Powered by Firebase):	9

Chapter 2: Functional & Non-Functional Requirements

2.1 - Functional and Conceptual Studies	10
2.1.1 - Functional Requirements:	10
2.1.1.1 - User Registration (Sign Up):	10
2.1.1.2 - User Authentication (Login):	10
2.1.1.3 - Authentication Methods:	10
2.1.1.5 - Search and Filtering:	10
2.1.1.6 - Dashboard and Analytics:	10
2.1.1.7 - Currency Toggle:	11
2.1.1.8 - Theme Toggle:	11
2.1.1.9 - PDF Export Functionality:	11
2.1.1.10 - Advanced Filters:	11
2.1.1.11 - Store Settings Panel:	11
2.1.1.12 - PDF Layout Customization:	11
2.1.1.13 - Task & Reminder Management:	11
2.1.1.14 - Account Settings:	12
2.1.1.15 - User Feedback:	12
2.1.1.16 - Logout:	12
2.1.2 - Non-Functional Requirements:	12
2.1.2.1 - Performance Requirements	12
2.1.2.2 - Scalability	12
2.1.2.3 - Availability	12
2.1.2.4 - Usability	13
2.1.2.5 - Security	13
2.1.2.6 - Maintainability	13
2.1.2.7 - Portability	13
2.1.2.8 - Localization & Currency Flexibility	13
2.1.2.9 - Data Backup & Recovery	13
2.1.2.10 - Compliance	14
2.2 - UML Use Case Diagram & Scenarios	14

2.2.1 -Use case diagram:.....	14
2.2.2 -Use case scenarios:.....	16

Chapter 3: System Design

3.1 - Entity Relationship Diagram:.....	24
---	----

Chapter 4: Interfaces & Pages Functionality

4.1 Walkthrough Page.....	26
4.2 Authentication Pages.....	27
4.3 Dashboard.....	28
4.4 Products Page.....	33
4.5 Customers Page.....	35
4.6 Carts Page.....	39
4.7 Sales Page.....	43
4.8 Settings Page.....	45
4.9 Tasks & Reminders.....	48
4.10 Feedback Page.....	50
4.11 Logout Function.....	51

Chapter 5: Closures

5.1 - Conclusion.....	52
5.2 - Self-Learned Outcomes.....	52
5.3 - Future Work.....	53
5.4 - References.....	54

List of Figures:

[Figure 1 - Use Case Diagram](#)

[Figure 2 - Entity Relationship Diagram](#)

[Figure 3 - Walkthrough UI](#)

[Figure 4 - Sign Up UI](#)

[Figure 5 - Login UI](#)

[Figure 6 - Dashboard 1](#)

[Figure 7 - Dashboard 2](#)

[Figure 8 - Dashboard 3](#)

[Figure 9 - Dashboard 4](#)

[Figure 10 - Products UI 1](#)

[Figure 11 - Products UI 2](#)

[Figure 12 - Customers UI 1](#)

[Figure 13 - Customers UI 2](#)

[Figure 14 - Customers UI 3](#)

[Figure 15 - Customers UI 4](#)

[Figure 16 - Carts UI 1](#)

[Figure 17 - Carts UI 2](#)

[Figure 18 - Carts UI 3](#)

[Figure 19 - Carts UI 5](#)

[Figure 20 - Sales UI 1](#)

[Figure 21 - Sales UI 2](#)

[Figure 22 - Store Settings UI](#)

[Figure 23 - PDF Layout Settings UI](#)

[Figure 24 - Account Settings UI](#)

[Figure 25 - Tasks & Reminders UI 1](#)

[Figure 26 - Tasks & Reminders UI 2](#)

[Figure 27 - Feedback UI](#)

[Figure 28 - Logout Function](#)

List Of Tables:

UC01 - Sign Up	19
UC02 - Sign In	19
UC03 - Browse Products	19
UC04 - Filter and Sort Products	20
UC05 - Export Products	20
UC06 - Search Product	20
UC07 - Edit Product	20
UC08 - Add Product	21
UC09 - Add Customer	21
UC10 - Browse Customers	21
UC11 - Search Customer	22
UC12 - Edit Customer & View Debts	22
UC13 - Create Cart	22
UC14 - Add Product to Cart	23
UC15 - Cancel Cart	23
UC16 - Add Sale	23
UC17 - Cancel Sale	24
UC18 - View Dashboard	24
UC19 - Export Dashboard Charts	24
UC20 - Submit Feedback	25
UC21 - Log Out	25

Chapter 1: Project Theory & Foundation

In this chapter, I am going to state exactly the problem I found and how this application helps in solving it. In addition I will mention the objective, scope and technology constraints of this application.

1.1 - Introduction:

Many small shops across Lebanon continue to rely on traditional pen-and-paper methods to record daily sales and track customer debts. This manual approach not only consumes time but also increases the risk of errors and data loss. Moreover, these businesses often struggle with basic inventory management—they lack visibility into which products are in stock, which are out of stock, and which items have remained unsold for extended periods. They are unable to identify their best-selling products or analyze sales trends, resulting in limited insight into their business performance. Overall, the absence of an organized system significantly hinders effective store and inventory management.

1.2 - Objective:

The primary objective of this project is to design and develop a modern, user-friendly Point of Sale (POS) system tailored for small retail businesses in Lebanon. This system aims to replace traditional manual methods with a digital solution that enables store owners to efficiently manage sales, track inventory in real time, monitor customer debts, and gain insights through sales analytics. By streamlining daily operations and improving accuracy, the POS system empowers shopkeepers to make informed decisions, reduce losses, and enhance overall business productivity.

1.3 - scope:

This project focuses on the development of a web-based Point of Sale (POS) system designed specifically for small and medium-sized retail businesses in Lebanon. The system is optimized for mobile use and supports essential store operations such as product management, sales processing, cart handling, customer debt tracking, and inventory monitoring. It also includes a dynamic dashboard that provides real-time insights into sales trends, stock levels, and business performance.

The POS system allows for multiple user accounts, enabling each store owner to manage their own data independently. Key features include product categorization, profit tracking, and PDF export functionality for reports and analytics. Additionally, the system supports customizable store settings such as currency, theme, and store name.

While the current scope is focused on core sales and inventory features, future enhancements may include advanced analytics, barcode scanning, adding products using CSV file, employee roles and permissions, and integration with payment gateways or accounting software. The

Chapter 1: Project Theory & Foundation

system is developed with scalability and usability in mind, ensuring it can grow alongside the needs of small businesses.

1.4 - Technology constraints:

- **Softwares:**
 - 1-Visual Studio Code – Source code editor
 - 2-Web Browser (Edge and Chrome) – For running and testing the application
 - 3-Firebase CLI – For deploying and managing Firebase services
- **Languages:**
 - 1-HTML5 – Structure and markup
 - 2-CSS3 – Styling and layout
 - 3-JavaScript (ES6+) – Application logic and interactivity
- **Frontend Technologies**
 - 1-Chart.js – Library for rendering responsive, interactive charts
 - 2-PDFMake – Client-side PDF generation for exporting reports
 - 3-SweetAlert2 – Beautiful, customizable popup and modal alerts
 - 4-Vanilla JavaScript (ES6+) – No frontend frameworks like React or Vue used; all logic written in native JavaScript
- **Backend / Cloud Services (Powered by Firebase)**
 - 1-Firebase Firestore – NoSQL cloud database for real-time data storage and sync
 - 2-Firebase Authentication – Secure user authentication (email, phone, Google sign-in)
 - 3-Firebase Hosting – Static site hosting with global CDN support

Chapter 2: Functional and Non-Functional Requirements

This chapter outlines the functional and non-functional requirements of the POS (Point of Sale) system application. It also presents the UML use case diagrams and corresponding scenarios to illustrate the user interactions with the system components in detail.

2.1 - Functional and Conceptual Studies

2.1.1 - Functional Requirements:

The system provides a comprehensive set of features tailored for small to medium-sized retail operations. The following are the key functional requirements:

2.1.1.1 - User Registration (Sign Up):

New users are prompted to create an account upon their first use of the application. An option is available to log in if the user already has an account.

2.1.1.2 - User Authentication (Login):

Registered users can securely log in to their account using multiple authentication methods.

2.1.1.3 - Authentication Methods:

The application supports login and sign-up using **email and password**, **phone number verification**, or **Google account integration**.

2.1.1.4 - Resource Management:

Users can **add**, **edit**, and **delete** essential resources including:

- Products (with details such as name, stock, cost price, barcode, image, etc.)
- Customers and their associated debts
- Carts and their product items
- Sales records and sales-related data

2.1.1.5 - Search and Filtering:

A dynamic search system enables users to locate products, customers, carts, or sales entries using input-based queries and filter options relevant to each section.

2.1.1.6 - Dashboard and Analytics:

A smart, real-time dashboard presents:

- Sales and profit insights
- Category distributions
- Inventory summaries
- Smart charts for visual analysis
- Comparisons (daily, weekly, monthly)

Chapter 2: Functional and Non-Functional Requirements

- Product lifecycle and popularity

2.1.1.7 - Currency Toggle:

The system allows users to switch between **US Dollar (USD)** and **Lebanese Pound (LBP)**. Prices and profits are automatically recalculated based on the selected currency and the configured exchange rate.

2.1.1.8 - Theme Toggle:

Users can switch between two predefined color theme combinations for a personalized user interface experience.

2.1.1.9 - PDF Export Functionality:

Users can generate and export **PDF reports** for:

- Products
- Customer debts
- Carts
- Sales documents
- Dashboard charts and summaries

2.1.1.10 - Advanced Filters:

Context-aware filters help users to narrow down information in each section, improving accessibility and efficiency.

2.1.1.11 - Store Settings Panel:

A dedicated settings section allows users to:

- Customize the store name
- Set the default currency and exchange rate
- Choose a preferred color combo for the interface

2.1.1.12 - PDF Layout Customization:

Users can personalize PDF export appearance, including:

- Title font and style
- Background color
- Even and odd row coloring

2.1.1.13 - Task & Reminder Management:

A dedicated section lets users create and manage tasks or reminders related to store operations.

Chapter 2: Functional and Non-Functional Requirements

2.1.1.14 - Account Settings:

Users can update personal account information such as:

- Username
- Email address
- Password
- Phone number

2.1.1.15 - User Feedback:

An integrated feedback system allows users to submit issues or suggestions, aiding in continuous improvement and user satisfaction.

2.1.1.16 - Logout:

Users can log out at any time, which safely ends the session and redirects them to the login screen.

2.1.2 - Non-Functional Requirements:

The non-functional requirements define the **quality attributes** of the POS system application. These requirements ensure the system is efficient, reliable, usable, and maintainable in real-world scenarios.

2.1.2.1 - Performance Requirements

- The application should load within **2 seconds** on a stable internet connection.
- Dashboard analytics and real-time charts must update without noticeable delays. Search and filtering operations should return results in **under 1 second**, even with a large dataset.

2.1.2.2 - Scalability

- The system should be able to handle a growing number of users, products, sales, and customer records without degradation in performance.
- Firebase's cloud infrastructure ensures horizontal scalability for both data and authentication services.

2.1.2.3 - Availability

- The application must be accessible **24/7** with a minimum uptime of **99.9%**, as guaranteed by Firebase Hosting.
- Offline fallback behavior may be implemented in future updates using Progressive Web App (PWA) support.

Chapter 2: Functional and Non-Functional Requirements

2.1.2.4 - Usability

- The interface is designed to be **mobile-first**, responsive, and intuitive for users with minimal technical skills.
- Tooltips, inline validations, and user feedback messages guide users through the app.
- All actions include confirmation steps or feedback to prevent user errors and confusion.

2.1.2.5 - Security

- User authentication is securely handled using **Firestore Authentication**, which supports secure password hashing, phone number verification, and OAuth2 via Google.
- Sensitive user data (email, password, phone number) is stored securely and never exposed on the frontend.
Firestore security rules restrict access to user-specific data, preventing unauthorized access.

2.1.2.6 - Maintainability

- The codebase is modular and organized using reusable components and Firebase services, making it easier to maintain and extend.
- The application logic is clearly separated from UI components to simplify debugging and future updates.

2.1.2.7 - Portability

- As a web-based solution, the application is accessible via modern browsers on **desktop**, **tablet**, and **mobile devices**, without requiring any installation.
- The codebase is compatible with deployment as a **PWA (Progressive Web App)** in future updates.

2.1.2.8 - Localization & Currency Flexibility

- The application supports toggling between **USD** and **LBP**, allowing it to adapt to local currency needs.
- Text and labels are written in English; future versions may include multi-language support based on user preferences.

2.1.2.9 - Data Backup & Recovery

- All data is stored in **Google Firestore**, which provides automatic backup and disaster recovery mechanisms.
- Real-time synchronization ensures that no critical data is lost during user interactions.

Chapter 2: Functional and Non-Functional Requirements

2.1.2.10 - Compliance

- The application follows best practices for user data protection and aligns with **general data privacy principles** (e.g., GDPR-like practices), although not officially certified.

2.2 - UML Use Case Diagram & Scenarios

2.2.1 -Use case diagram:

The use case diagram provides a high-level visual representation of the main interactions between the user (actor) and the POS system (system boundary). It outlines the core functionalities that the user can perform, helping to define the system's behavior from an end-user perspective.

In this POS system application, the primary actor is the User, which represents store owners or staff members who interact with the application. See [Figure 1](#)

Chapter 2: Functional and Non-Functional Requirements

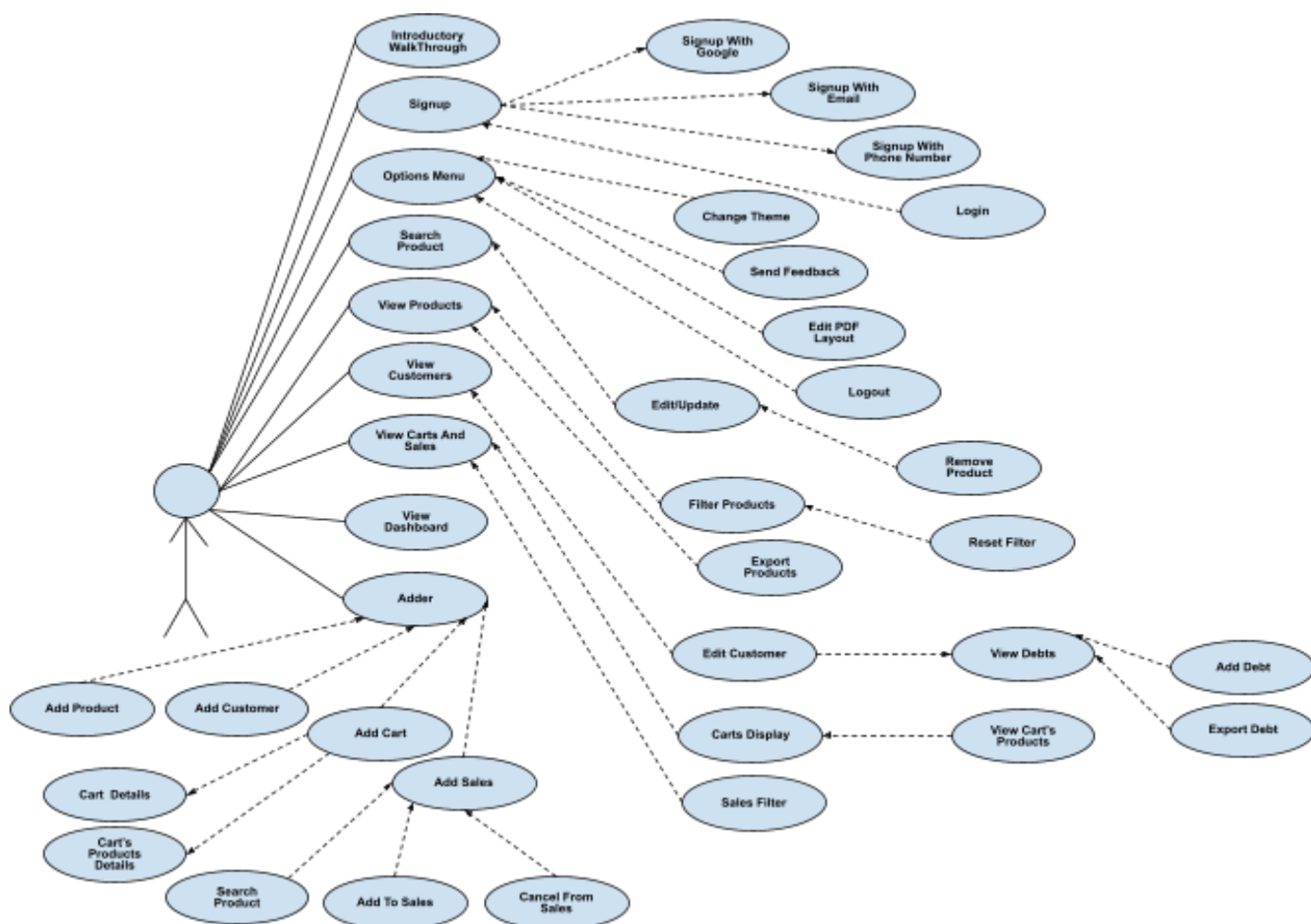


Figure 1 - Use Case Diagram

Chapter 2: Functional and Non-Functional Requirements

2.2.2 -Use case scenarios:

Use case scenarios provide detailed step-by-step narratives of how the user interacts with the system to accomplish specific goals. Each scenario focuses on a particular functionality and describes the flow of events, including user actions and system responses.

In this POS system application, use case scenarios illustrate common operations such as:

- **Signing Up and Logging In** using email, phone number, or Google account
- **Adding a new product** to the inventory
- **Creating a sales transaction** from selected cart items
- **Filtering and exporting sales reports**
- **Viewing insights on the dashboard**
- **Updating store settings and account information**
- **Submitting feedback or logging out**

Each scenario typically includes:

- **Actor:** The user interacting with the system
- **Preconditions:** What must be true before the scenario begins
- **Main Flow:** The regular sequence of steps the user follows
- **Alternative/Exception Flows:** Variations, errors, or system validations that might occur

These scenarios help ensure that the system is designed with all possible user interactions in mind and serve as the foundation for testing and validation of the application features.

Chapter 2: Functional and Non-Functional Requirements

UC01 - Sign Up

Primary Actor	User
Goal	Create a new account using email, Google, or phone number.
Preconditions	App is launched, and user is on authentication screen.
Postconditions	A new user account is created and user is logged in.
Main Flow	1. User selects 'Sign Up'. 2. User chooses a method (Email, Google, or Phone Number). 3. System validates input and creates a user account. 4. User is redirected to the main app interface.
Alternative Flow	AF1: If the user enters an invalid email or password, an error message is shown and authentication is blocked. AF2: If third-party authentication (Google/Phone) fails due to network or provider issues, the user is informed and allowed to retry. AF3: If required fields are left empty, the user is prompted to complete them before proceeding.

UC02 - Sign In

Primary Actor	User
Goal	Authenticate and access the application.
Preconditions	Account already exists.
Postconditions	User is authenticated and inside the application.
Main Flow	1. User selects 'Sign In'. 2. User enters credentials or uses Google/Phone Number. 3. System verifies credentials. 4. User is logged in and redirected to the main interface.
Alternative Flow	AF1: If the user enters an invalid email or password, an error message is shown and authentication is blocked. AF2: If third-party authentication (Google/Phone) fails due to network or provider issues, the user is informed and allowed to retry. AF3: If required fields are left empty, the user is prompted to complete them before proceeding.

UC03 - Browse Products

Primary Actor	User
Goal	View all available products.
Preconditions	User is logged in.
Postconditions	Products are displayed.
Main Flow	1. User lands on the Products section. 2. System displays all user-specific products with stock, cost, etc.

Chapter 2: Functional and Non-Functional Requirements

UC04 - Filter and Sort Products

Primary Actor	User
Goal	Filter/sort product list by various criteria.
Preconditions	User is viewing the Products section.
Postconditions	Filtered/sorted product list is shown.
Main Flow	1. User applies filters (category, stock range, profit, etc.). 2. User can sort results by profit, stock, or price. 3. System displays matching products.

UC05 - Export Products

Primary Actor	User
Goal	Export current list of products.
Preconditions	User is viewing the Products section.
Postconditions	PDF of product data is generated.
Main Flow	1. User clicks 'Export Products'. 2. System generates a PDF of current product data. 3. PDF is available for download or sharing.

UC06 - Search Product

Primary Actor	User
Goal	Search for a product by name/label.
Preconditions	User is viewing the Products section.
Postconditions	Matching results are displayed.
Main Flow	1. User types product name in search bar. 2. System displays matching results dynamically.
Alternative Flow	AF1: If the search yields no results, the system displays “No products found”.

UC07 - Edit Product

Primary Actor	User
Goal	Update product information.
Preconditions	User is viewing a product.
Postconditions	Product is updated and saved.
Main Flow	1. User clicks a product. 2. Edit form appears with product fields. 3. User modifies fields and confirms. 4. System validates and saves changes.

Chapter 2: Functional and Non-Functional Requirements

UC08 - Add Product

Primary Actor	User
Goal	Add a new product to the inventory.
Preconditions	User is logged in.
Postconditions	Product is added to inventory.
Main Flow	1. User opens the adder section → clicks 'Add Product'. 2. System shows form with input validation. 3. User fills and submits. 4. System adds product to their collection.
Alternative Flow	AF1: If the product name, barcode, or costPrice field is invalid (e.g., empty or negative), the system shows validation errors and blocks submission. AF2: If a product with the same label or barcode already exists, the user is prompted to edit the existing product instead. AF3: If the user cancels before submitting, the form is cleared and not saved.

UC09 - Add Customer

Primary Actor	User
Goal	Add a customer to the system.
Preconditions	User is logged in.
Postconditions	Customer is added.
Main Flow	1. User clicks 'Add Customer'. 2. User enters name and phone number. 3. System saves customer entry.
Alternative Flow	AF1: If the name or phone number is invalid or missing, the user is prompted to correct it. AF2: If a customer with the same name exists, a confirmation is shown before allowing duplicate creation.

UC10 - Browse Customers

Primary Actor	User
Goal	View all added customers.
Preconditions	User is logged in.
Postconditions	Customer list is displayed.
Main Flow	1. User opens the Customers section. 2. System displays customer cards with name & phone number.

Chapter 2: Functional and Non-Functional Requirements

UC11 - Search Customer

Primary Actor	User
Goal	Search customer by name.
Preconditions	Customers are added.
Postconditions	Matching customers are shown.
Main Flow	1. User types in search bar. 2. System returns matching customers.
Alternative Flow	AF1: If the search yields no results, the system displays “No customers found.”

UC12 - Edit Customer & View Debts

Primary Actor	User
Goal	Update customer info and view their debt details.
Preconditions	Customer exists.
Postconditions	Customer data is updated; PDF optionally generated.
Main Flow	1. User clicks on a customer card. 2. System shows form to update name/phone + display debt details. 3. Total balance is displayed. 4. User can export the debt details as PDF.

UC13 - Create Cart

Primary Actor	User
Goal	Create a new cart.
Preconditions	User is logged in.
Postconditions	Cart is created.
Main Flow	1. User opens Adder → clicks 'Add Cart'. 2. System prompts for cart name (required). 3. Cart is created.
Alternative Flow	AF1: If the user leaves the cart name empty, a required field message is displayed. AF2: If no matching product is found when searching, a “Product not found” message is shown. AF3: If a product is added but stock is insufficient, an alert is shown and the item is not added. AF4: If the cart is canceled before checkout, all added product stocks are restored and the cart is deleted.

Chapter 2: Functional and Non-Functional Requirements

UC14 - Add Product to Cart

Primary Actor	User
Goal	Add items to the created cart.
Preconditions	Cart is created.
Postconditions	Item added to cart and sales recorded.
Main Flow	1. User searches for a product. 2. System checks for match and displays product. 3. User adds to cart. 4. Product stock is reduced, and a sale record is created.

UC15 - Cancel Cart

Primary Actor	User
Goal	Cancel an active cart.
Preconditions	User has an active cart.
Postconditions	Cart and items are removed, stock restored.
Main Flow	1. User clicks 'Cancel Cart'. 2. System restores inventory levels and removes the cart.

UC16 - Add Sale

Primary Actor	User
Goal	Add a product directly to sales.
Preconditions	User is logged in.
Postconditions	Sale added and stock updated.
Main Flow	1. User opens Adder → clicks 'Add Sale'. 2. User searches product → adds to sales. 3. Stock is reduced, sale recorded.
Alternative Flow	AF1: If the user tries to add a product without enough stock, an error is shown.

Chapter 2: Functional and Non-Functional Requirements

UC17 - Cancel Sale

Primary Actor	User
Goal	Cancel a sale and restore stock.
Preconditions	Sale has been added.
Postconditions	Sale removed and stock restored.
Main Flow	1. User cancels a product in the sales section. 2. System restores product stock and removes sale record.
Alternative Flow	AF1: If a sale is canceled, the product stock is incremented accordingly. AF2: If no products are available, the sale form remains inactive or prompts the user to add products first.

UC18 - View Dashboard

Primary Actor	User
Goal	View all analytics and summaries.
Preconditions	User is logged in.
Postconditions	Analytics displayed.
Main Flow	1. User opens Dashboard section. 2. System displays all charts and summaries.
Alternative FLOW	AF1: If there's no data for the selected time range, the charts show "No data available." AF2: If the user selects an invalid date range (e.g., end date before start date), a warning is shown.

UC19 - Export Dashboard Charts

Primary Actor	User
Goal	Export all analytics charts.
Preconditions	Dashboard is accessible.
Postconditions	PDF of charts is generated.
Main Flow	1. User clicks export. 2. System generates PDF containing all current charts.

Chapter 2: Functional and Non-Functional Requirements

UC20 - Submit Feedback

Primary Actor	User
Goal	Submit feedback and screenshots.
Preconditions	User is logged in.
Postconditions	Feedback is saved to database.
Main Flow	1. User opens menu → clicks 'Feedback'. 2. System shows form for text input + optional screenshot. 3. Feedback is saved in database linked to user ID.
Alternative Flow	AF1: If the screenshot upload fails, the user is notified and allowed to proceed with text-only feedback. AF2: If the feedback message is empty, submission is blocked and a prompt appears.

UC21 - Log Out

Primary Actor	User
Goal	Sign out of the application.
Preconditions	User is logged in.
Postconditions	User is signed out.
Main Flow	1. User opens menu → clicks 'Logout'. 2. System signs user out and redirects to sign-in screen.

Chapter 3: System Design

This chapter describes the system design, including the database, the related tables and the relations between them.

3.1 - Entity Relationship Diagram:

The ER (Entity-Relationship) Diagram illustrates the data structure and relationships between key entities in the POS (Point of Sale) system application. It provides a visual representation of how data is organized, stored, and connected across different sections of the application. Using **Firebase** as a backend utility.

The main entities include:

- **User:** Represents the account holder with fields such as `userId`, `email`, `phone`, and `userName`.
- **Product:** Stores product details like `productId`, `label`, `costPrice`, `stock`, `category`, `barcode` and `createdAt`.
- **Customer:** Contains customer data including `name`, and `phoneNumber`.
- **Debt:** A sub-entity of Customer that tracks outstanding balances and includes fields like `balance`, `createdAt`, `details` and `totalBalance`.
- **Cart:** Represents a temporary collection of products with `cartId`, `dateCreated`, `name`, and `totalCost`.
- **CartProduct:** A sub-entity of Cart that stores each product in the cart along with `quantity`, `costPrice`, and `total`.
- **SoldProduct:** A sub-entity of Sale that stores data for each sold product with fields such as `quantity`, `profit`, and `total`.
- **Sale:** Documents daily transactions and includes fields like `dateSold`, `totalProfit`, `totalRevenue`, and `totalProductsSold`.

Each entity is connected using one-to-many or one-to-one relationships. For example:

- One **Customer** can have many **Debts**.
- One **Cart** can contain multiple **CartProducts**.
- One **Sale** can include multiple **SoldProducts**.

Chapter 3: System Design

This ER diagram ensures the database structure is logical, normalized, and optimized for efficient data retrieval and updates within the POS application. See [Figure 2](#)

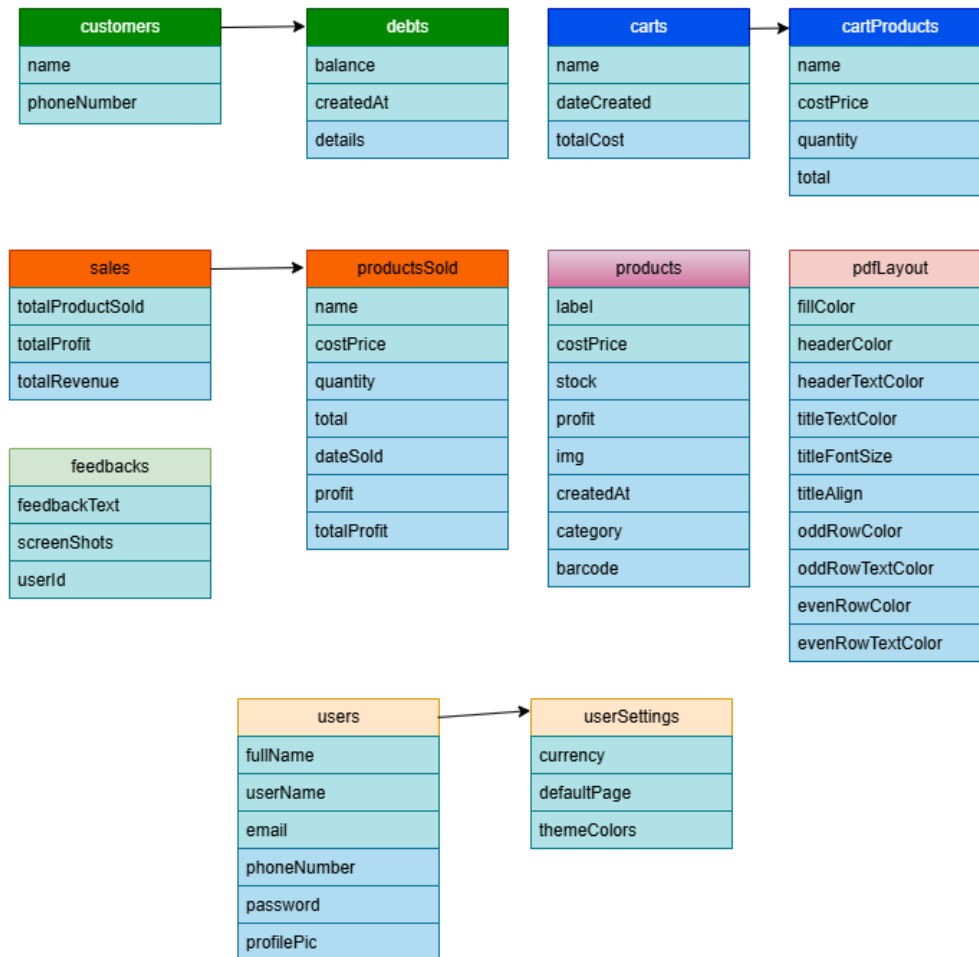


Figure 2 - Entity Relationship Diagram

Chapter 4: Interfaces & Pages Functionality

This chapter presents a comprehensive overview of the application's user interface. It highlights the design structure and explains the purpose and functionality of each main page, section, and interactive element within the system.

4.1 Walkthrough Page

Introduces new users to the application's core features with a clean, guided onboarding experience. Helps users understand the system before signing up. See [Figure 3](#)

Figure 3 - Walkthrough UI



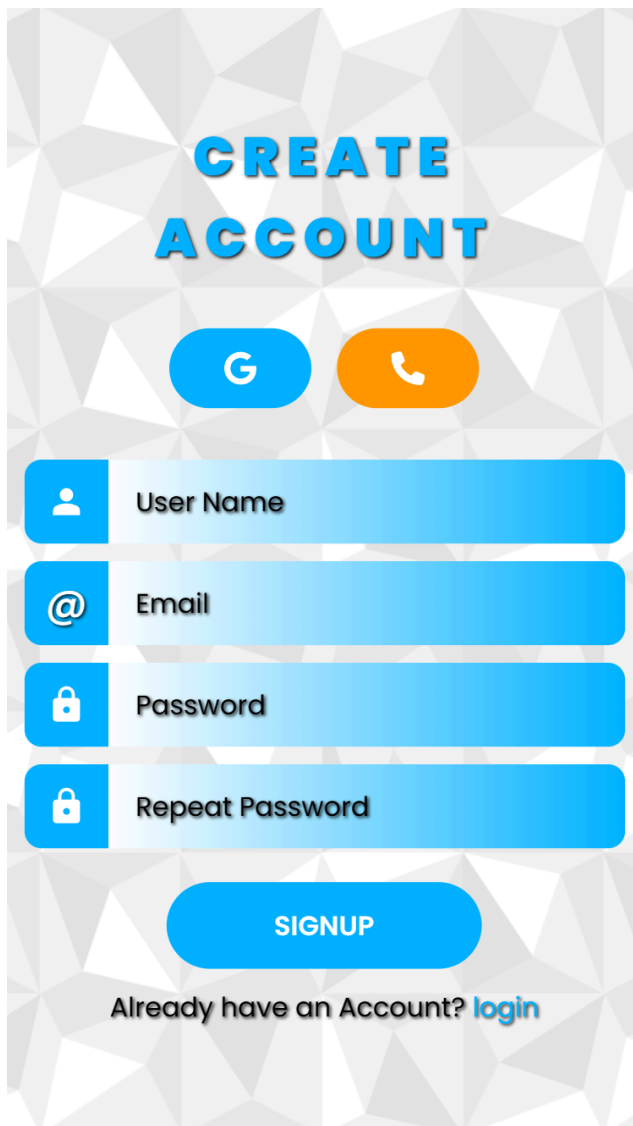
Chapter 4: Interfaces & Pages Functionality

4.2 Authentication Pages

Includes:

- **Sign-Up Page:** Allows users to register via email, phone number, or Google account. See [Figure 4](#)
- **Login Page:** Provides secure access to existing accounts. See [Figure 5](#)
- **Signup & Login Methods:** Offers options to choose the preferred registration method.

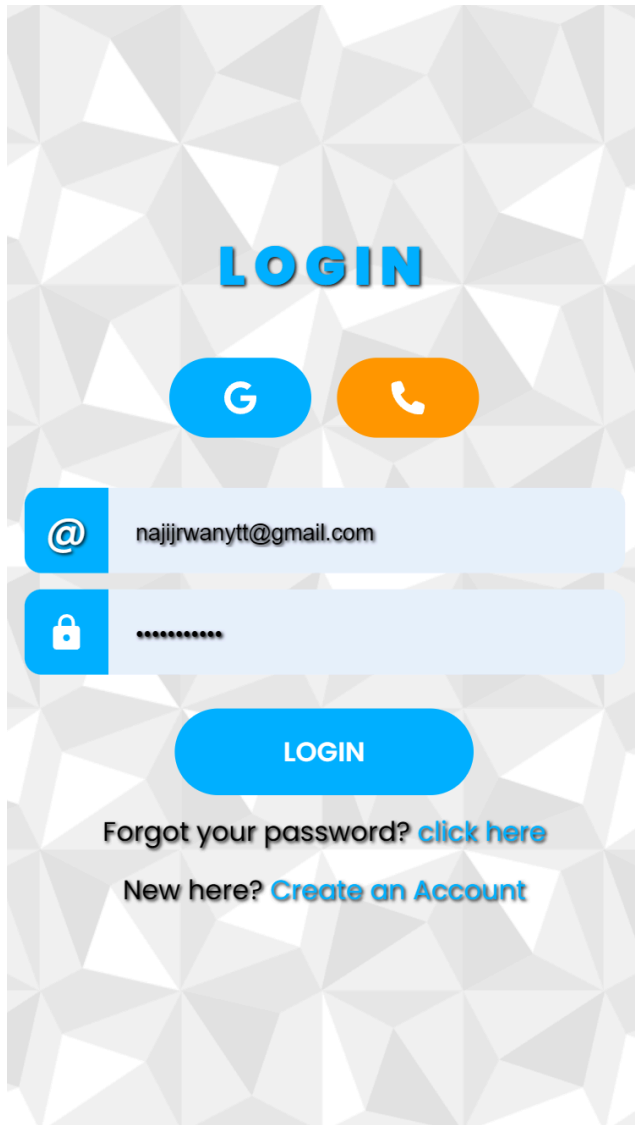
Figure 4 - Sign Up UI



The image shows a mobile app sign-up interface with a light gray geometric background. At the top, the text "CREATE ACCOUNT" is displayed in large, bold, blue capital letters. Below this, there are two rounded rectangular buttons: a blue one with a white "G" icon and an orange one with a white phone icon. Underneath these are four input fields, each with a blue header bar containing an icon and a light blue body. The first field has a person icon and is labeled "User Name". The second has an "@" icon and is labeled "Email". The third has a lock icon and is labeled "Password". The fourth has a lock icon and is labeled "Repeat Password". At the bottom, there is a large blue rounded button labeled "SIGNUP" in white capital letters. Below the button, the text "Already have an Account? [login](#)" is displayed, with "login" in blue.

Chapter 4: Interfaces & Pages Functionality

Figure 5 - Login UI

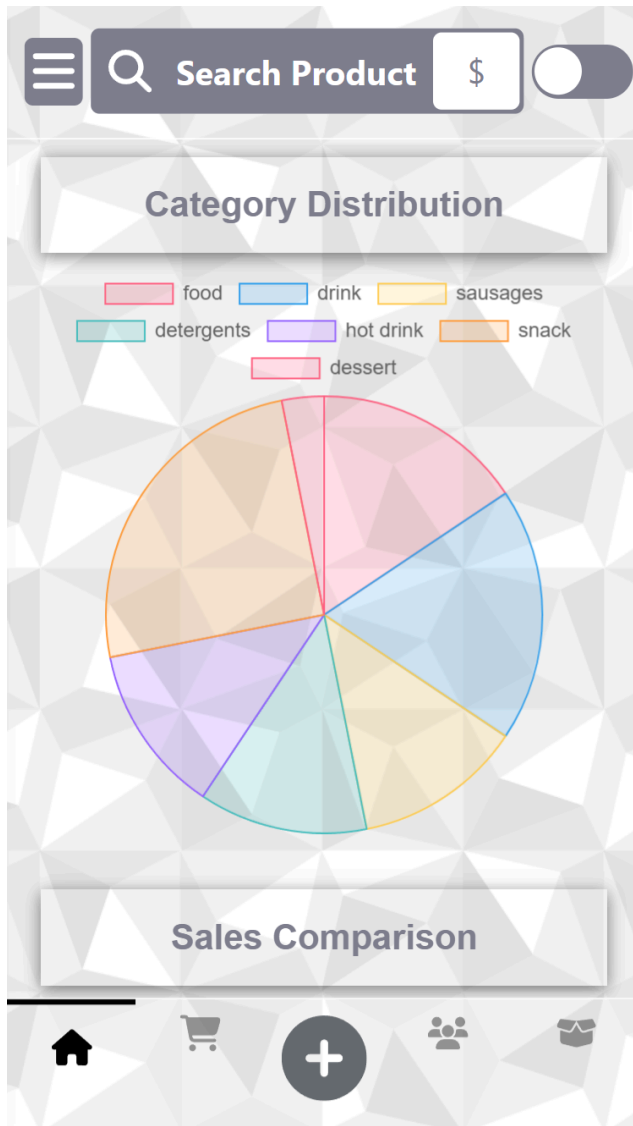


4.3 Dashboard

Chapter 4: Interfaces & Pages Functionality

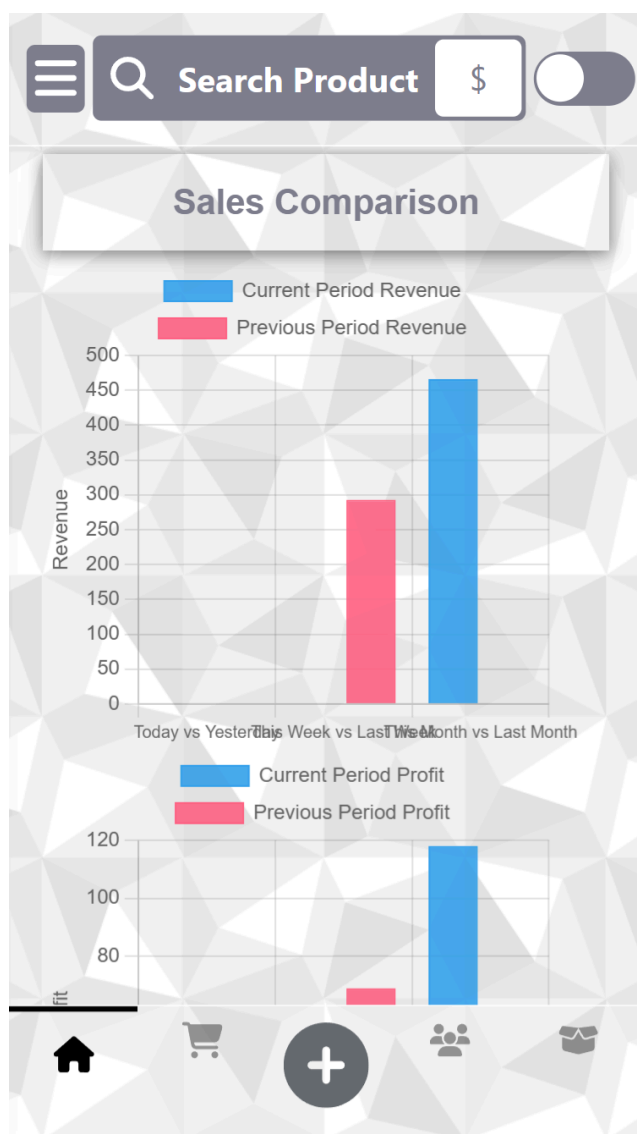
Displays real-time charts, smart insights, and inventory summaries. Allows filtering by date and exporting charts to PDF. Key metrics include sales trends, category breakdowns, low stock alerts, and profit margins. See figures [Figure 6](#) - [Figure 7](#) - [Figure 8](#) - [Figure 8](#)

Figure 6 - Dashboard 1



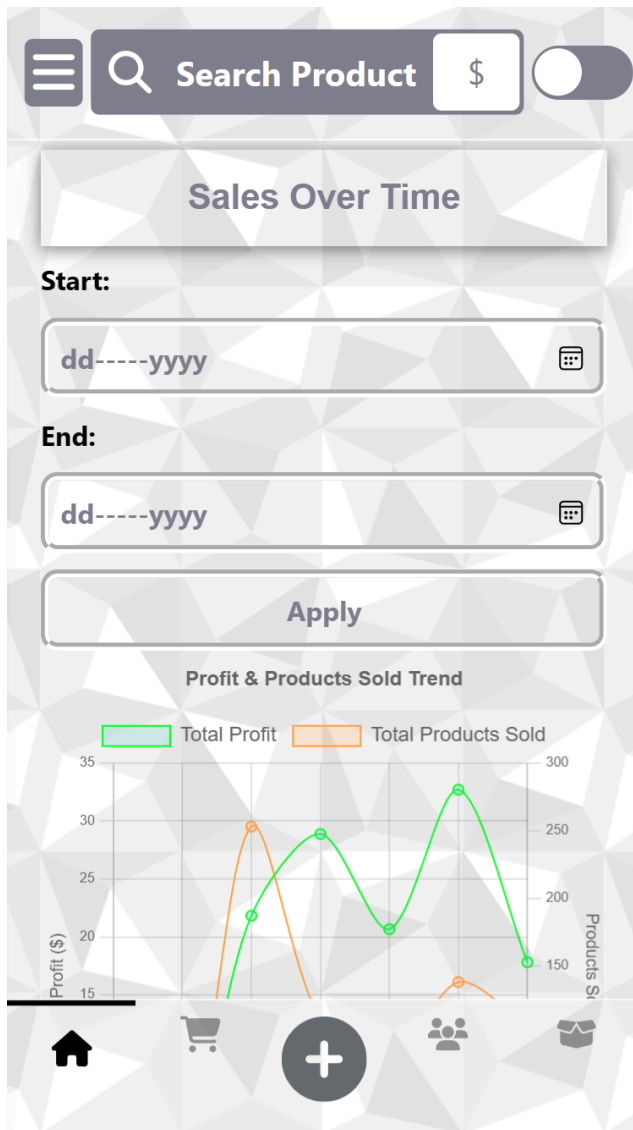
Chapter 4: Interfaces & Pages Functionality

Figure 7 - Dashboard 2



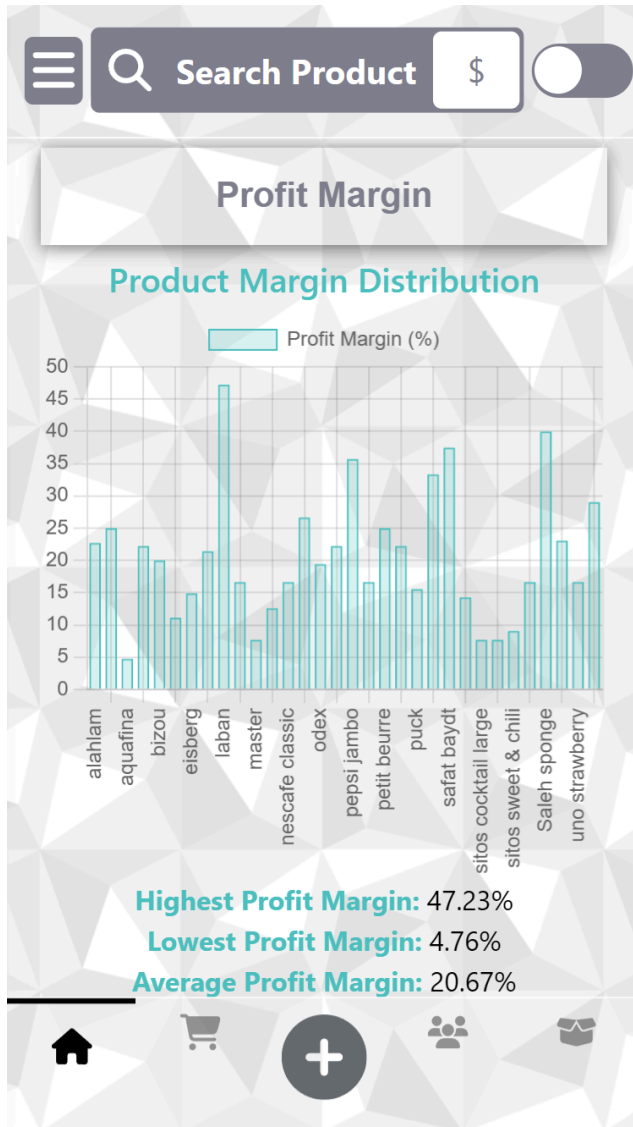
Chapter 4: Interfaces & Pages Functionality

Figure 8 - Dashboard 3



Chapter 4: Interfaces & Pages Functionality

Figure 9 - Dashboard 4

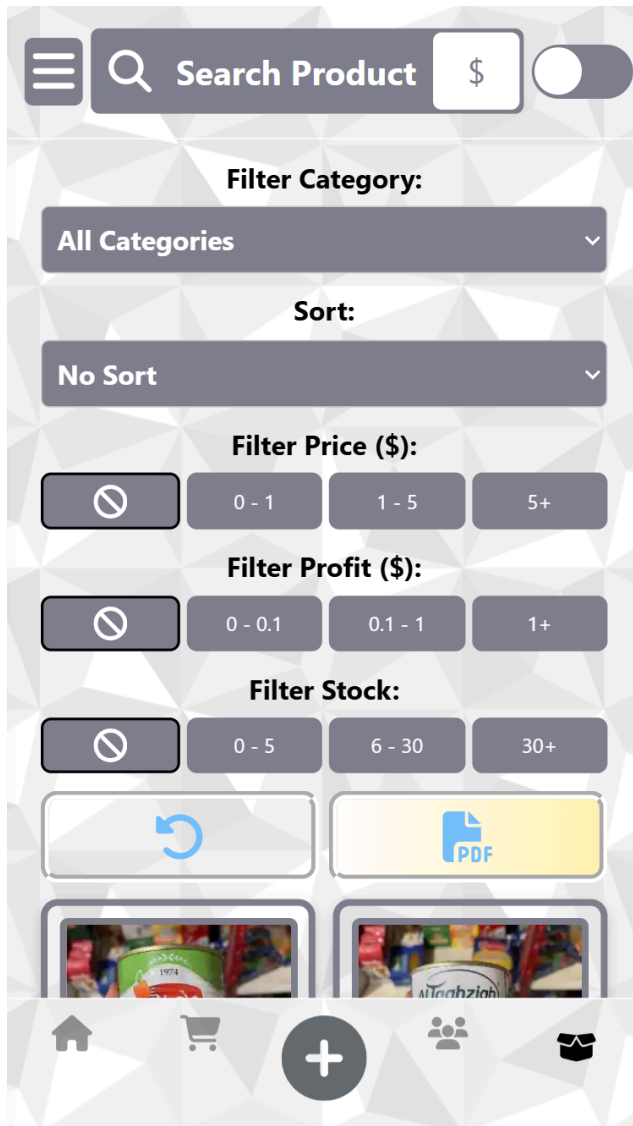


Chapter 4: Interfaces & Pages Functionality

4.4 Products Page

Allows users to add, view, edit, and delete products. Displays essential product data like name, stock, cost price, barcode, and category. Supports filtering and PDF export. See [Figure 10](#) and [Figure 11](#)

Figure 10 - Products UI 1



Chapter 4: Interfaces & Pages Functionality

Figure 11- Products UI 2



Chapter 4: Interfaces & Pages Functionality

4.5 Customers Page

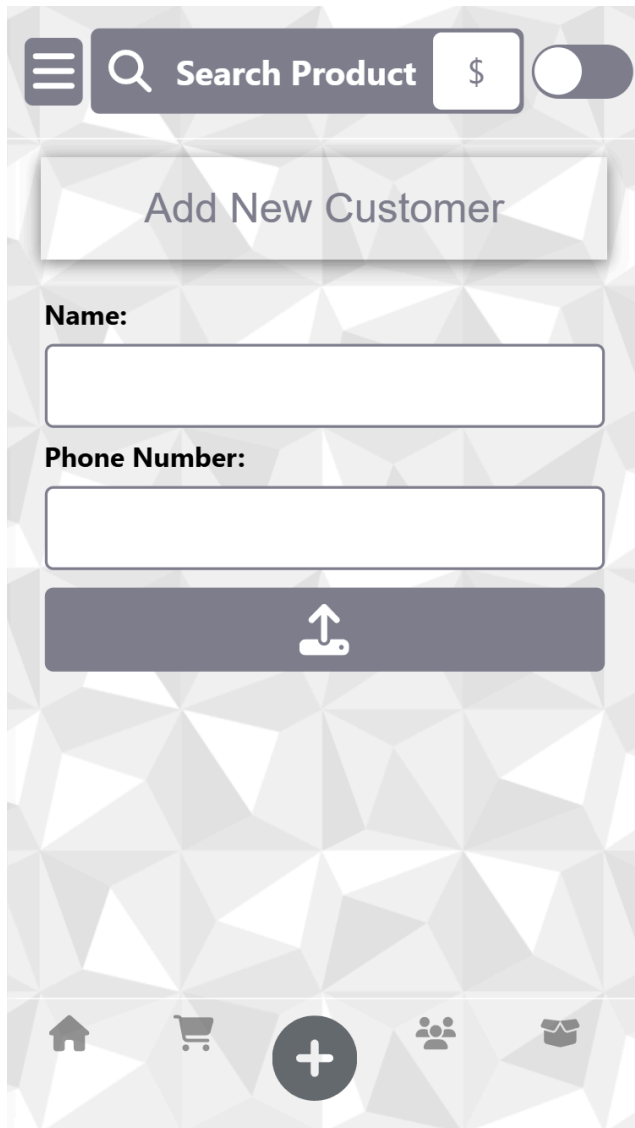
Manages customer profiles and debts (See [Figure 12](#)). Users can add new customers (See [Figure 13](#)), record outstanding balances (See [Figure 14](#)), and update customer information (See [Figure 15](#)). Includes export features and search filters.

Figure 12 - Customers UI 1



Chapter 4: Interfaces & Pages Functionality

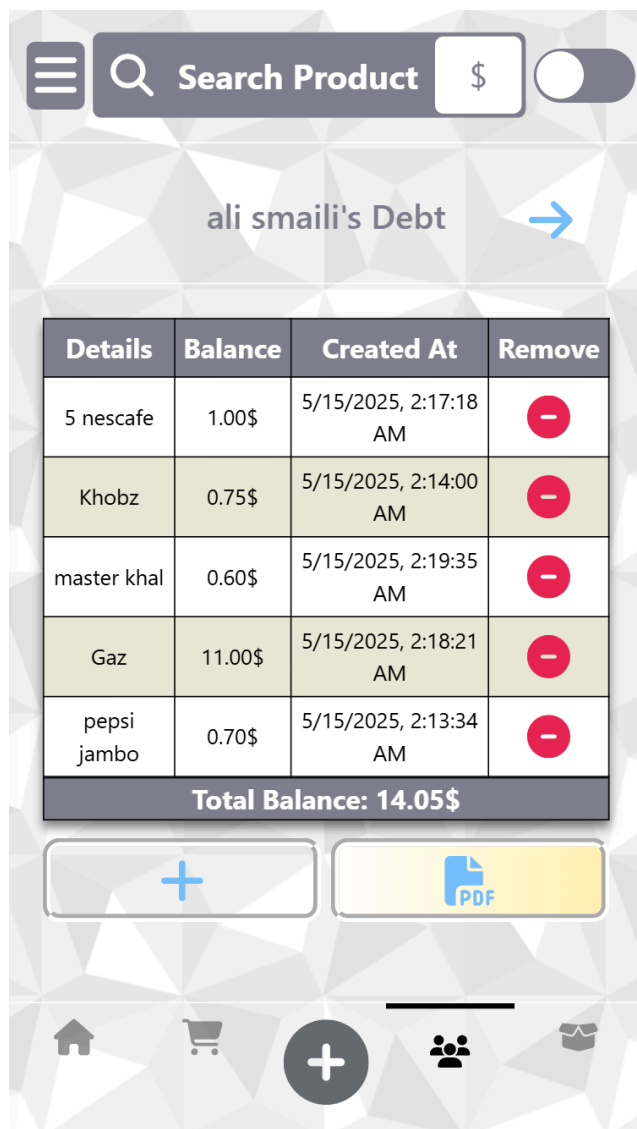
Figure 13 - Customers UI 2



The image shows a mobile application interface for adding a new customer. At the top, there is a dark grey header bar containing a hamburger menu icon, a search bar with the text "Search Product", a dollar sign icon, and a toggle switch. Below the header is a large, light grey button with the text "Add New Customer". Underneath this button are two text input fields. The first is labeled "Name:" and the second is labeled "Phone Number:". Below the input fields is a dark grey button with a white upload icon (an arrow pointing up from a document). At the bottom of the screen is a navigation bar with five icons: a home icon, a shopping cart icon, a central plus icon, a group of people icon, and a storefront icon. The background of the app has a subtle geometric pattern.

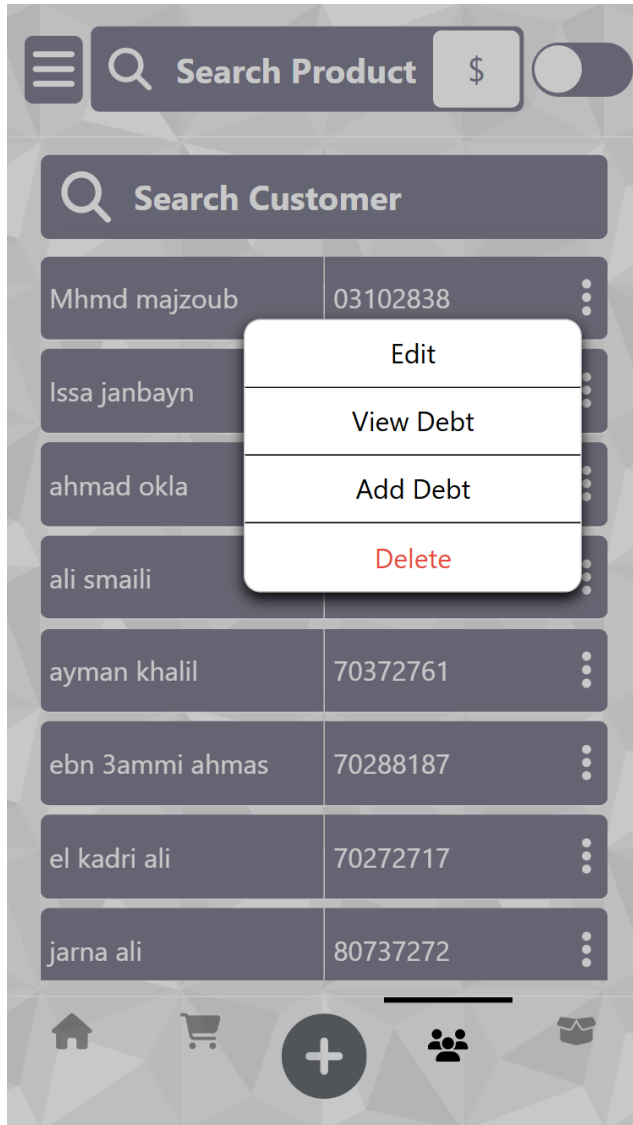
Chapter 4: Interfaces & Pages Functionality

Figure 14 - Customers UI 3



Chapter 4: Interfaces & Pages Functionality

Figure 15 - Customers UI 4

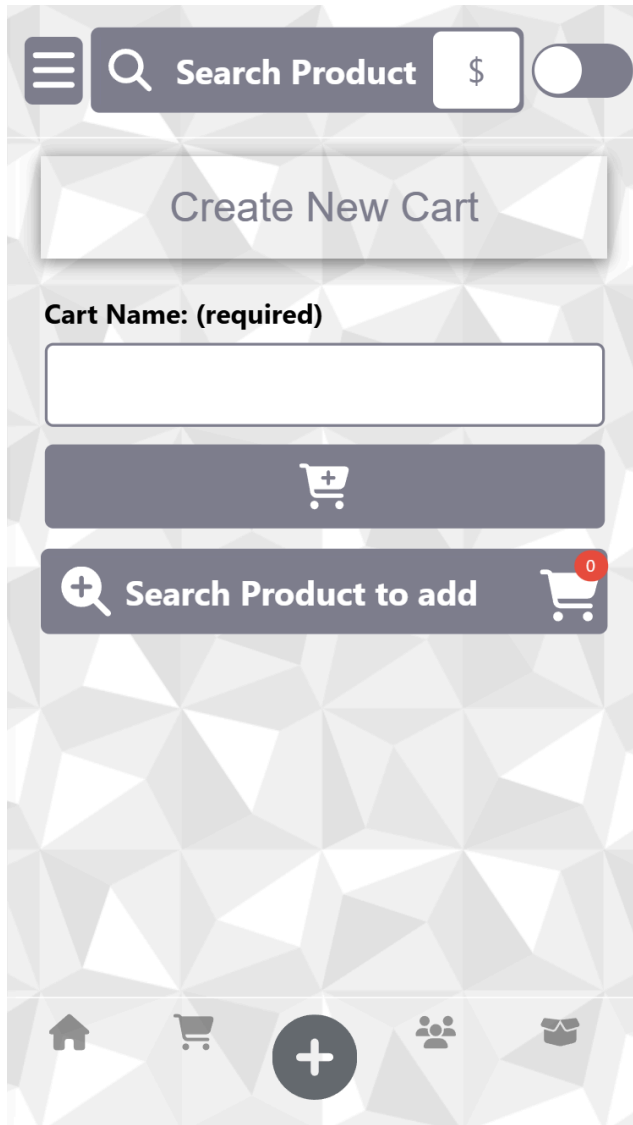


Chapter 4: Interfaces & Pages Functionality

4.6 Carts Page

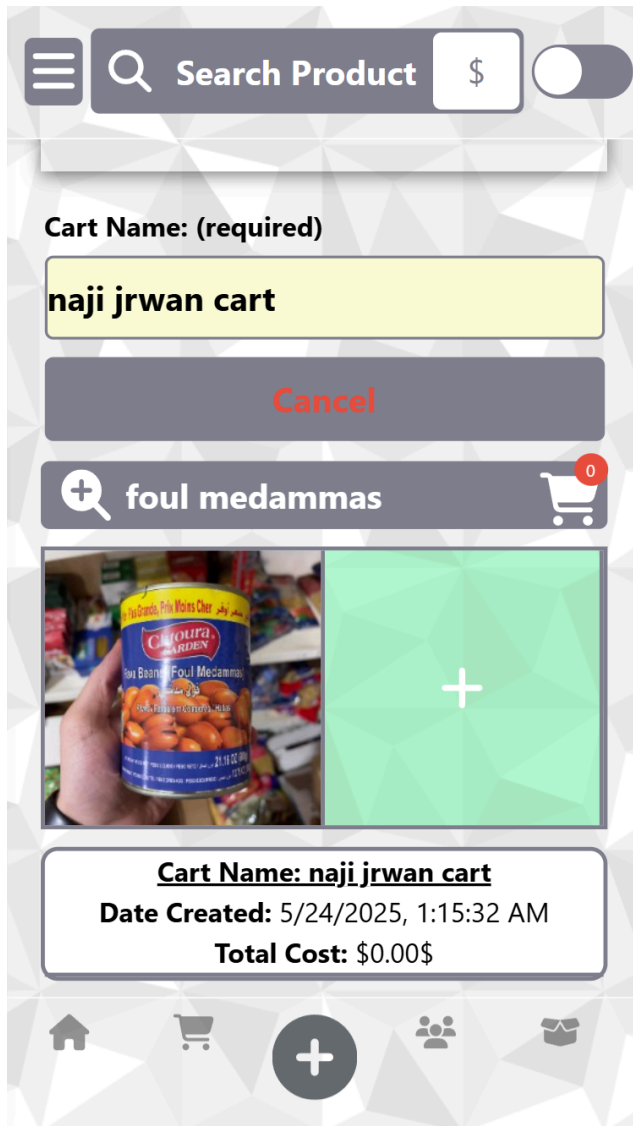
Users can create and manage carts with selected products (See [Figure 16](#) and [Figure 17](#)). Useful for managing ongoing sales before final checkout. Each cart displays product quantities, total cost, and allows editing or deletion (See [Figure 18](#) and [Figure 19](#)).

Figure 16 - Carts UI 1



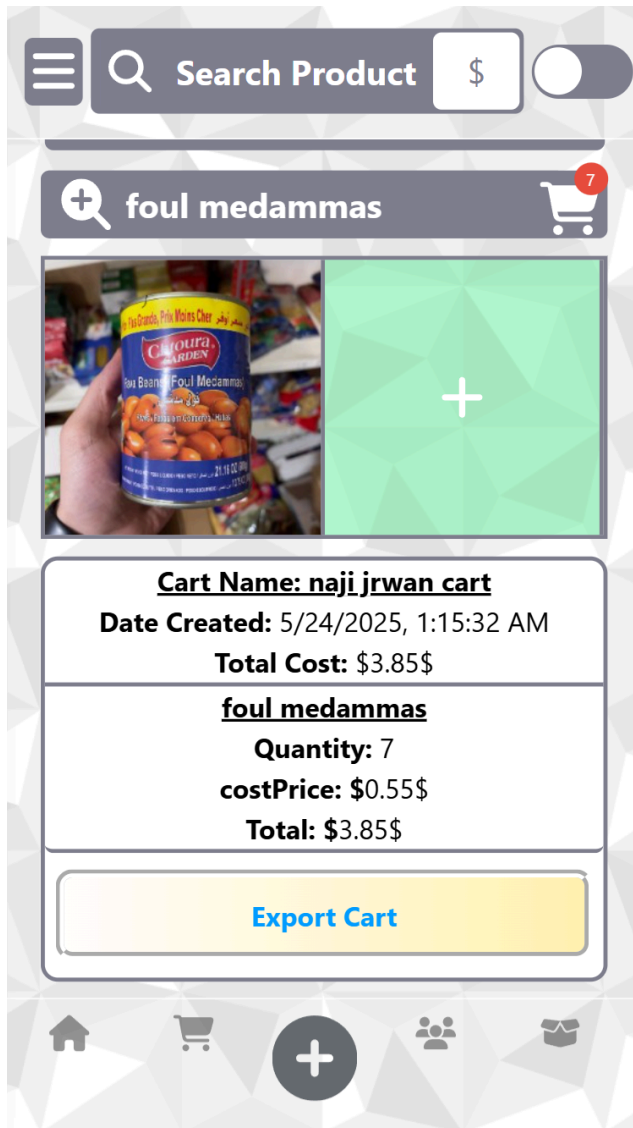
Chapter 4: Interfaces & Pages Functionality

Figure 17 - Carts UI 2



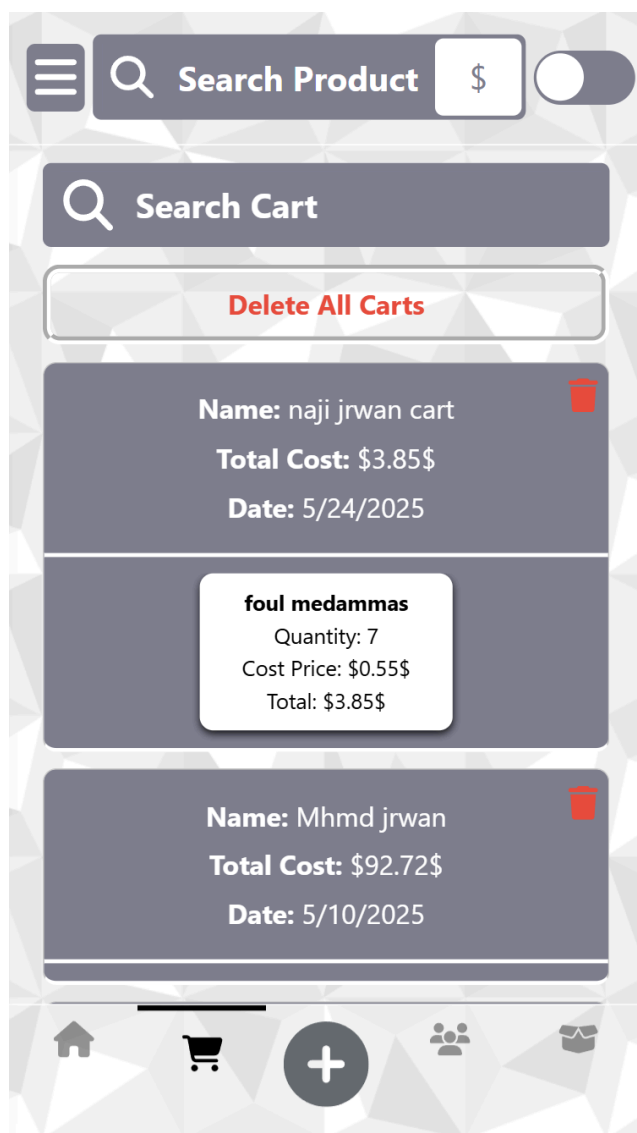
Chapter 4: Interfaces & Pages Functionality

Figure 18 - Carts UI 3



Chapter 4: Interfaces & Pages Functionality

Figure 19 - Carts UI 5

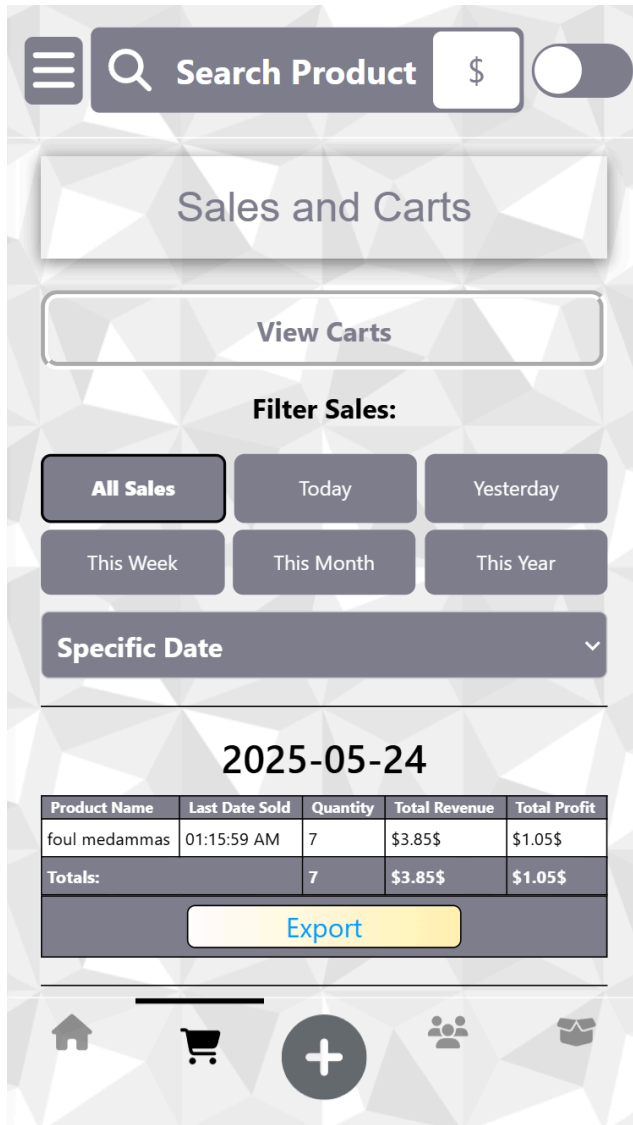


Chapter 4: Interfaces & Pages Functionality

4.7 Sales Page

Displays daily recorded sales with details like quantity sold, total revenue, and profit. Allows filtering by date and viewing specific sold products. Sales data can be exported to PDF for reports. See [Figure 20](#) and [Figure 21](#)

Figure 20 - Sales UI 1



Chapter 4: Interfaces & Pages Functionality

Figure 21 - Sales UI 2



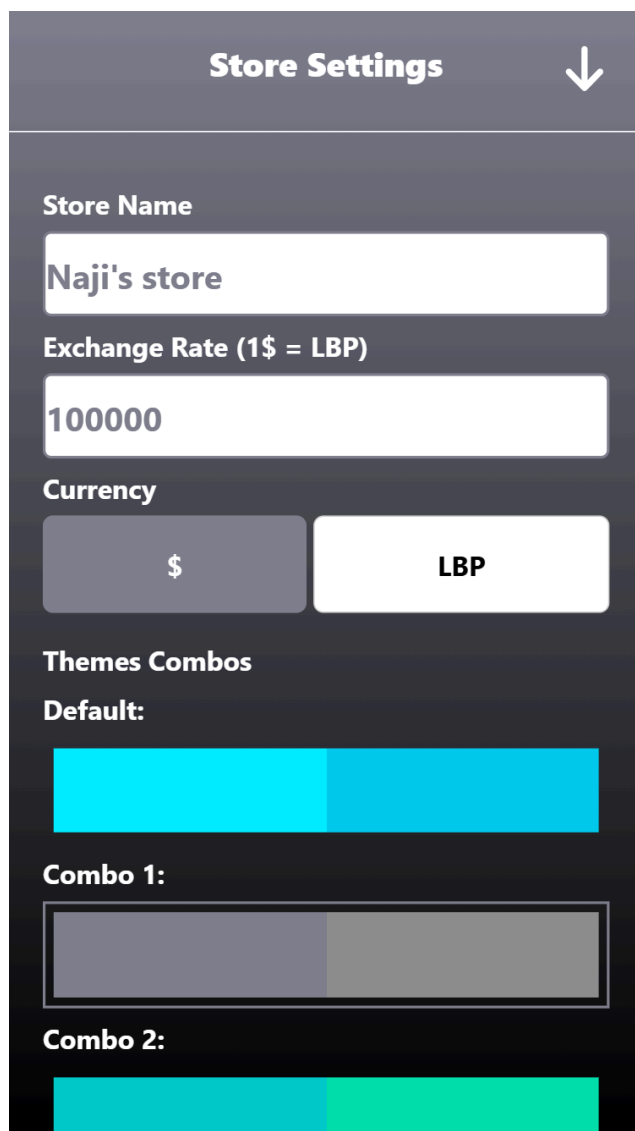
Chapter 4: Interfaces & Pages Functionality

4.8 Settings Page

Includes:

- **Store Settings:** Set store name, default currency, exchange rate, and color themes. See [Figure 22](#)
- **PDF Layout Settings:** Customize exported PDF appearance (background, titles, row colors). See [Figure 23](#)
- **Account Settings:** Update user info like email, phone number, and password. See [Figure 24](#)

Figure 22 - Store Settings UI



The screenshot displays the 'Store Settings' interface. At the top, a dark header bar contains the title 'Store Settings' and a downward-pointing arrow. Below this, the settings are organized into sections. The 'Store Name' section features a text input field with the value 'Naji's store'. The 'Exchange Rate (1\$ = LBP)' section has a text input field showing '100000'. The 'Currency' section includes two buttons: a grey one with a '\$' symbol and a white one with 'LBP'. The 'Themes Combos' section is divided into three parts: 'Default:' with a cyan color bar, 'Combo 1:' with a grey color bar, and 'Combo 2:' with a teal color bar.

Chapter 4: Interfaces & Pages Functionality

Figure 23 - PDF Layout Settings UI

Change PDF Layout ↓

Background Color:
[White color selection box]

Title Text Color:
[Black color selection box]

Title Font Size:
[16 text input box]

Title Alignment:
☒ Left ☐ Center ☐ Right

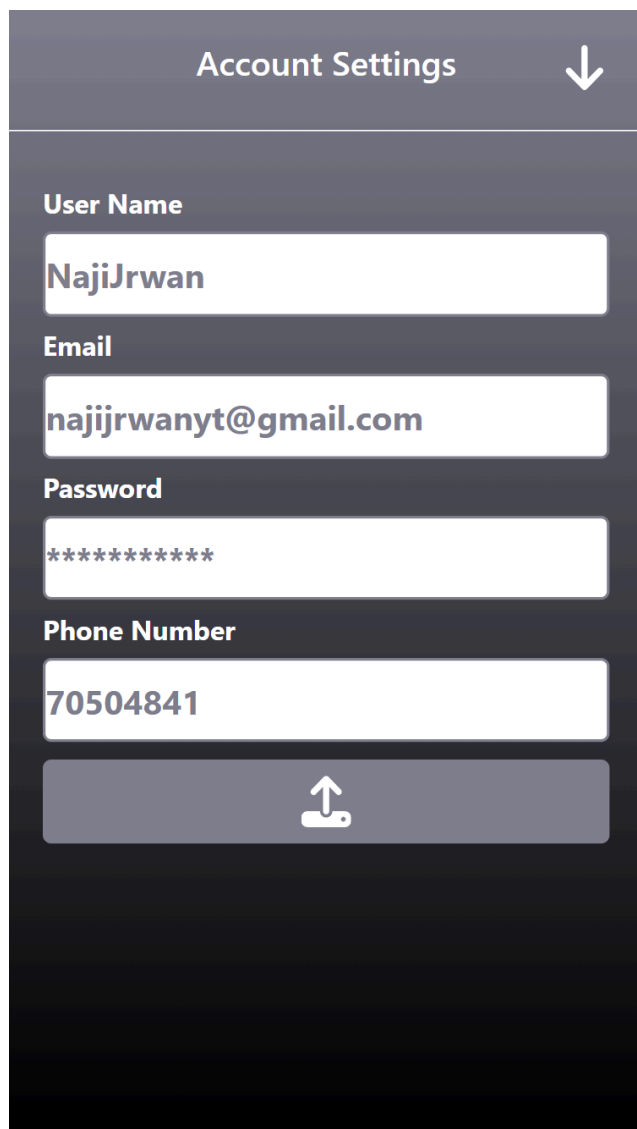
Header Row Color:
[Blue color selection box]

Header Text Color:
[White color selection box]

Even Row Color:
[Light blue color selection box]

Chapter 4: Interfaces & Pages Functionality

Figure 24 - Account Settings UI



The image shows a mobile application interface for 'Account Settings'. At the top, there is a dark header bar with the text 'Account Settings' and a white downward-pointing arrow icon. Below the header, the form is organized into sections. The 'User Name' section has a white text input field containing 'NajiJrwan'. The 'Email' section has a white text input field containing 'najiJrwanyt@gmail.com'. The 'Password' section has a white text input field with masked characters '*****'. The 'Phone Number' section has a white text input field containing '70504841'. At the bottom of the form, there is a wide, dark button with a white upward-pointing arrow icon, likely for saving or submitting the changes.

Chapter 4: Interfaces & Pages Functionality

4.9 Tasks & Reminders

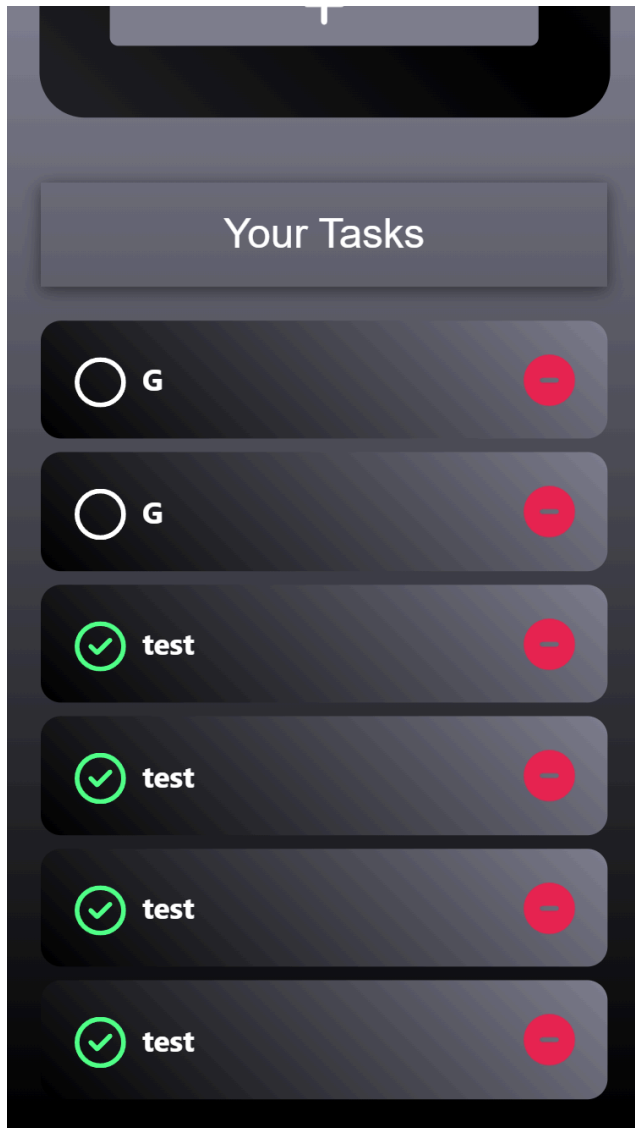
Allows users to set daily tasks and reminders to stay organized and manage responsibilities within the business. See [Figure 25](#) and [Figure 26](#)

Figure 25 - Tasks & Reminders UI 1

The screenshot shows a mobile application interface for 'Tasks & Reminders'. At the top, there is a header bar with the title 'Tasks & Reminders' and a downward-pointing arrow icon. Below the header, the main content area is dark-themed and contains a rounded rectangle with the title 'Add New Task'. Inside this rectangle, there are three input fields: 'Task title', 'Task details (optional)', and 'Due Date & Time (optional):'. The 'Due Date & Time' field has a placeholder 'dd----yyyy --:-- --' and a calendar icon. Below these fields is a large button with a white plus sign. At the bottom of the screen, there is a section titled 'Your Tasks'.

Chapter 4: Interfaces & Pages Functionality

Figure 26 - Tasks & Reminders UI 2

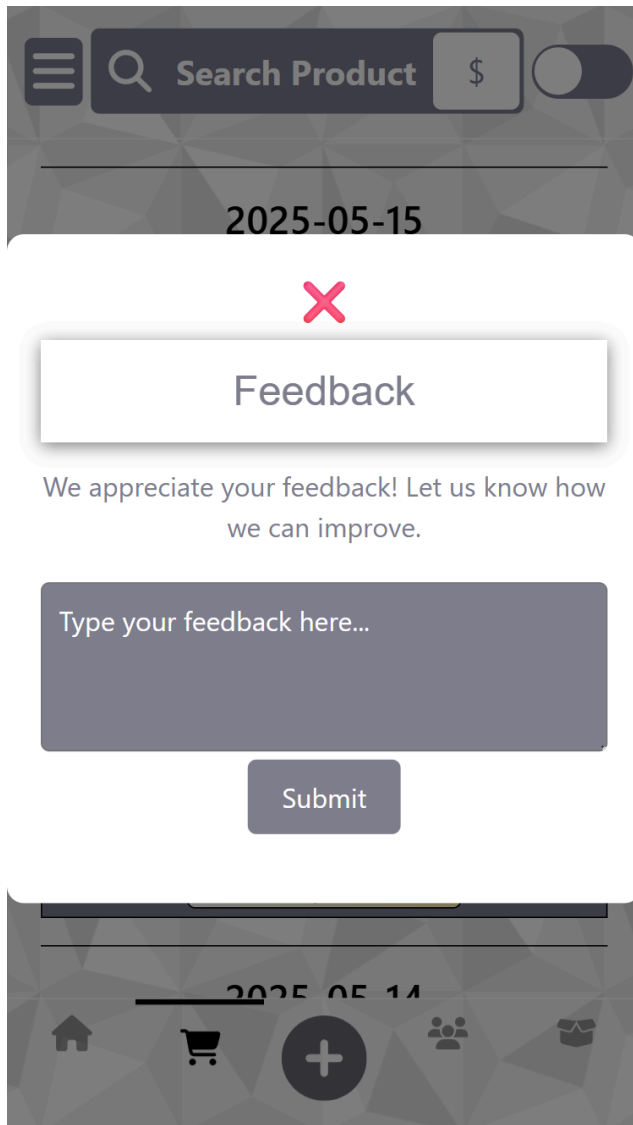


Chapter 4: Interfaces & Pages Functionality

4.10 Feedback Page

Enables users to submit feedback or report issues. See [Figure 27](#)

Figure 27 - Feedback UI

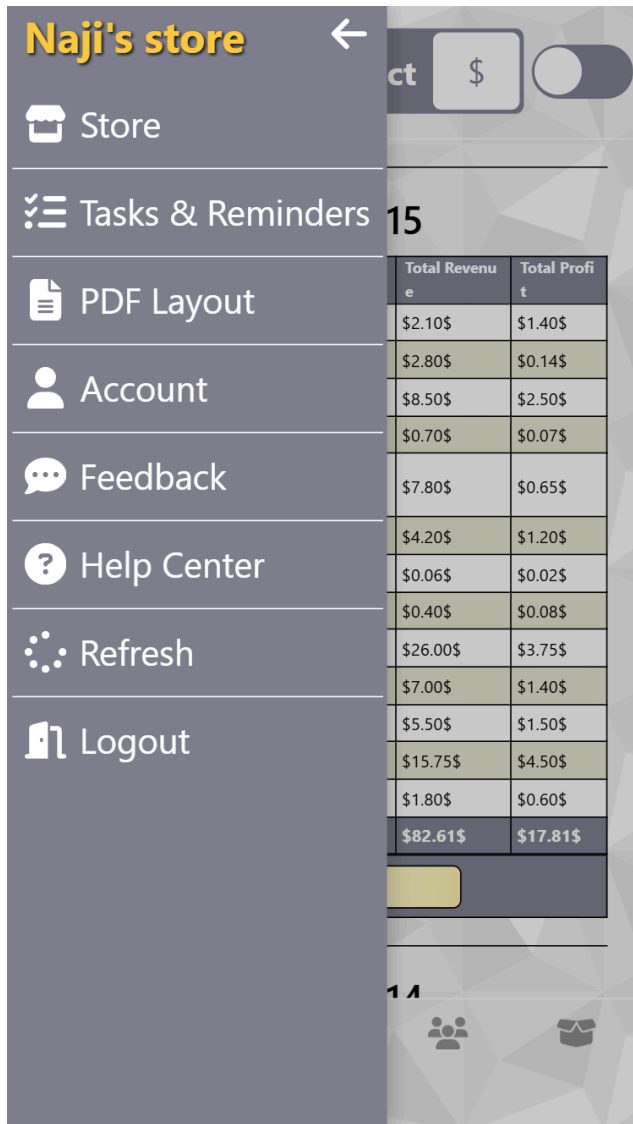


Chapter 4: Interfaces & Pages Functionality

4.11 Logout Function

Securely logs the user out and redirects them to the login page. See [Figure 28](#)

Figure 28 - Logout Function



Chapter 5: Closures

This final chapter presents a comprehensive conclusion to the work conducted throughout this study. It reflects on the main outcomes and achievements, summarizes the key findings, and evaluates how the objectives were met. Additionally, this chapter outlines the self-learned outcomes gained during the development and research process, providing personal insights into the learning journey.

Furthermore, it discusses potential future work and improvements that can be made to extend or enhance the current project. This includes suggestions for technical enhancements, research directions, and possible real-world applications. Lastly, the chapter concludes with a list of references that have supported the work, offering a foundation for further exploration and study.

5.1 - Conclusion

The primary aim of this project was to design and implement a mobile-first Point of Sale (POS) web application that supports multi-user access, real-time sales tracking, inventory management, customer debt tracking, and dynamic reporting. Through careful planning, structured development, and consistent iteration, these objectives were achieved with a strong emphasis on usability, data accuracy, and performance.

The system now includes features such as product management, sales logging, smart dashboard analytics, cart functionality, customer debt handling, and customizable exports — all tied together with a responsive, user-friendly interface. The integration with Firebase Firestore allowed real-time data updates and secure storage, while custom PDF exports and chart visualizations brought clarity to user insights. Every component was designed with flexibility and scalability in mind, laying a strong foundation for future improvements.

This project successfully demonstrates how modern web technologies can be used to create a powerful business tool that simplifies daily operations, enhances data visibility, and supports informed decision-making.

5.2 - Self-Learned Outcomes

This project was also a rich personal learning journey, covering a wide range of both technical and non-technical skills. Among the most significant technical takeaways were:

- **Mastering Firebase Firestore:** From basic document management to advanced queries and real-time listeners, I learned to structure collections efficiently and optimize reads and writes.
- **Asynchronous JavaScript:** Through trial, error, and refinement, I developed a deeper understanding of `async/await`, especially in scenarios involving Firestore calls, authentication, and chained updates.

Chapter 5: Closures

- **PDF and Chart Exports:** Customizing PDF layouts and dynamically exporting chart data honed my skills in programmatic document generation and improved my attention to detail in data presentation.
- **Modular Thinking:** Organizing logic in a scalable way using utility files, inline modals, and modular function calls made the codebase easier to extend, especially as more features were added.

On a personal level, I learned the importance of **patience**, **problem decomposition**, and **incremental improvement**. I also gained confidence in asking the right questions and breaking down complex problems into manageable parts — a crucial skill in software development.

5.3 - Future Work

Although the current version of the POS system is fully functional and meets its intended goals, there are several areas where it could be further expanded or enhanced:

- **Advanced Role-Based Access Control (RBAC):** Currently, the system supports multiple users, but future work could include fine-grained permissions to distinguish between managers, cashiers, and admins.
- **Automated Low-Stock Alerts:** Implementing notification logic to automatically inform users of low or out-of-stock items would streamline inventory control.
- **AI-driven Sales Insights:** Incorporating machine learning to detect sales trends, forecast demand, or suggest restocking strategies based on past data.
- **Multi-language Support:** Adding language localization to make the app usable for a wider demographic.
- **Stripe/PayPal Integration:** Enabling online payments would make the app more competitive and valuable for small businesses that sell both online and in-store.
- **CSV Import/Export:** Currently, exports are focused on PDF. Adding CSV or Excel support would allow users to work with external accounting tools more easily.

Each of these features builds upon the existing foundation and would provide tangible benefits to users while pushing the project further into real-world readiness.

Chapter 5: Closures

5.4 - References

This project relied on a variety of official documentation, libraries, and online resources that provided both technical guidance and implementation support. Below is a list of key references and tools used throughout the development:

1. **Firebase Documentation**
<https://firebase.google.com/docs>
Used extensively for Firestore, Authentication, and Hosting integration.
2. **Chart.js Documentation**
<https://www.chartjs.org/docs/latest/>
Used for rendering dynamic charts in the dashboard analytics section.
3. **jspdf & html2canvas GitHub Repositories**
 - <https://github.com/parallax/jsPDF>
 - <https://github.com/niklasvh/html2canvas>
Used to generate custom PDF exports for products, carts, debts, and sales reports.
4. **JavaScript Info**
<https://javascript.info/>
A helpful learning resource for improving async/await usage and modular JavaScript development.
5. **MDN Web Docs**
<https://developer.mozilla.org/>
Used for general JavaScript, HTML, and browser-related references.
6. **Stack Overflow Discussions**
Various threads were referenced to debug Firestore rules, optimize queries, and troubleshoot PDF issues.
7. **Google Fonts & Material Icons**
Integrated for consistent typography and iconography across the app interface.

These resources were vital to overcoming development challenges and ensuring a clean, responsive, and feature-rich user experience.

This concludes the project. Thank you for reaching the final page, and for following the journey from idea to implementation :)